Project 3 - Econ 144

Mason Lee

2025-12-04

Introduction

For this project, we will be looking at FRED's DJFUELUSGULF.csv dataset, which tracks daily prices of Kerosene-Type Jet Fuel (USD per barrel) in the Gulf Coast. It is not seasonally adjusted, and prices vary anywhere from below 50 cents to around $4.50 in the 35+ years since the first observation was taken (April 1990).

This dataset is interesting because fuel prices can vary for many different reasons. Aside from inflation, which accounts for the general upwards trend, seasonal and cyclical trends occur. The data is also sourced from the Gulf Coast, which has been historically unstable. That can lead to price spikes that will be hard to forecast.

The only two columns in this dataset are "Date" and "Price", and there are a few missing values from over the years, but due to the patterns, which will be shown in the next section, the data should prove interesting to work with and forecast.

Results

Packages

```
library(readr)
library(lattice)
library(foreign)
library(MASS)
library(car)
```

```
## Loading required package: carData
```

```
require(stats)
require(stats4)
```

```
## Loading required package: stats4
```

```
library(KernSmooth)
```

```
## KernSmooth 2.23 loaded
## Copyright M. P. Wand 1997-2009
```

```
library(fastICA)
library(cluster)
library(leaps)
library(mgcv)
```

```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(rpart)
library(pan)
library(mgcv)
library(DAAG)
```

```
##
## Attaching package: 'DAAG'
```

```
## The following object is masked from 'package:car':
##
##     vif
```

```
## The following object is masked from 'package:MASS':
##
##     hills
```

```
library("TTR")
library(tis)
```

```
##
## Attaching package: 'tis'
```

```
## The following object is masked from 'package:TTR':
##
##     lags
```

```
## The following object is masked from 'package:mgcv':
##
##     ti
```

```
require("datasets")
require(graphics)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
##
## Attaching package: 'forecast'
```

```
## The following object is masked from 'package:tis':
##
##     easter

## The following object is masked from 'package:nlme':
##
##     getResponse
```

```
require(astsa)
```

```
## Loading required package: astsa

##
## Attaching package: 'astsa'

## The following object is masked from 'package:forecast':
##
##     gas
```

```
library(RColorBrewer)
library(plotrix)
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr

##
## Attaching package: 'tsibble'

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, union
```

```
library(ggplot2)
library(tidyr)
library(USgas)
library(fpp3)
```

```
## -- Attaching packages --------------------------------------- fpp3 1.0.2 --

## v tibble      3.2.1     v tsibbledata 0.4.1
## v dplyr       1.1.4     v feasts      0.4.2
## v lubridate   1.9.4     v fable       0.4.1

## -- Conflicts ------------------------------------------------ fpp3_conflicts --
## x feasts::ACF()         masks nlme::ACF()
## x dplyr::between()      masks tis::between()
## x dplyr::collapse()     masks nlme::collapse()
## x lubridate::date()     masks base::date()
```

```
## x lubridate::day()      masks tis::day()
## x dplyr::filter()       masks stats::filter()
## x lubridate::hms()      masks tis::hms()
## x tsibble::intersect()  masks base::intersect()
## x lubridate::interval() masks tsibble::interval()
## x dplyr::lag()          masks stats::lag()
## x lubridate::month()    masks tis::month()
## x lubridate::period()   masks tis::period()
## x lubridate::POSIXct()  masks tis::POSIXct()
## x lubridate::quarter()  masks tis::quarter()
## x dplyr::recode()       masks car::recode()
## x dplyr::select()       masks MASS::select()
## x tsibble::setdiff()    masks base::setdiff()
## x lubridate::today()    masks tis::today()
## x tsibble::union()      masks base::union()
## x lubridate::year()     masks tis::year()
## x lubridate::ymd()      masks tis::ymd()
```

```
library(fable.prophet)
```

```
## Loading required package: Rcpp
```

```
library(fabletools)
library(feasts)
```

Data Cleaning

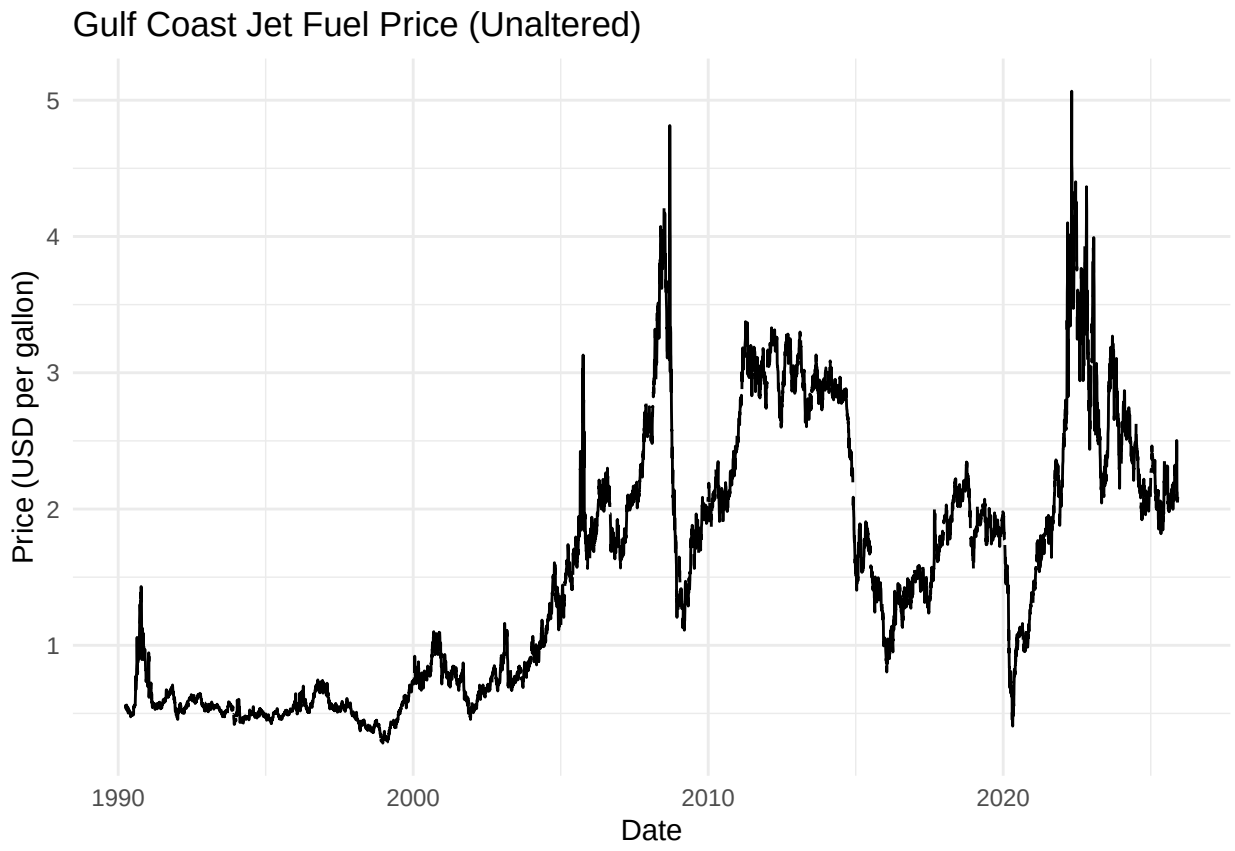
```
set.seed(817)

fuel_raw <- read_csv("DJFUELUSGULF.csv")
```

```
## Rows: 9306 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (1): DJFUELUSGULF
## date (1): observation_date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(fuel_raw)
```

```
## # A tibble: 6 x 2
##   observation_date DJFUELUSGULF
##   <date>                  <dbl>
## 1 1990-04-02              0.55
## 2 1990-04-03              0.555
## 3 1990-04-04              0.56
## 4 1990-04-05              0.54
## 5 1990-04-06              0.536
## 6 1990-04-09              0.536
```

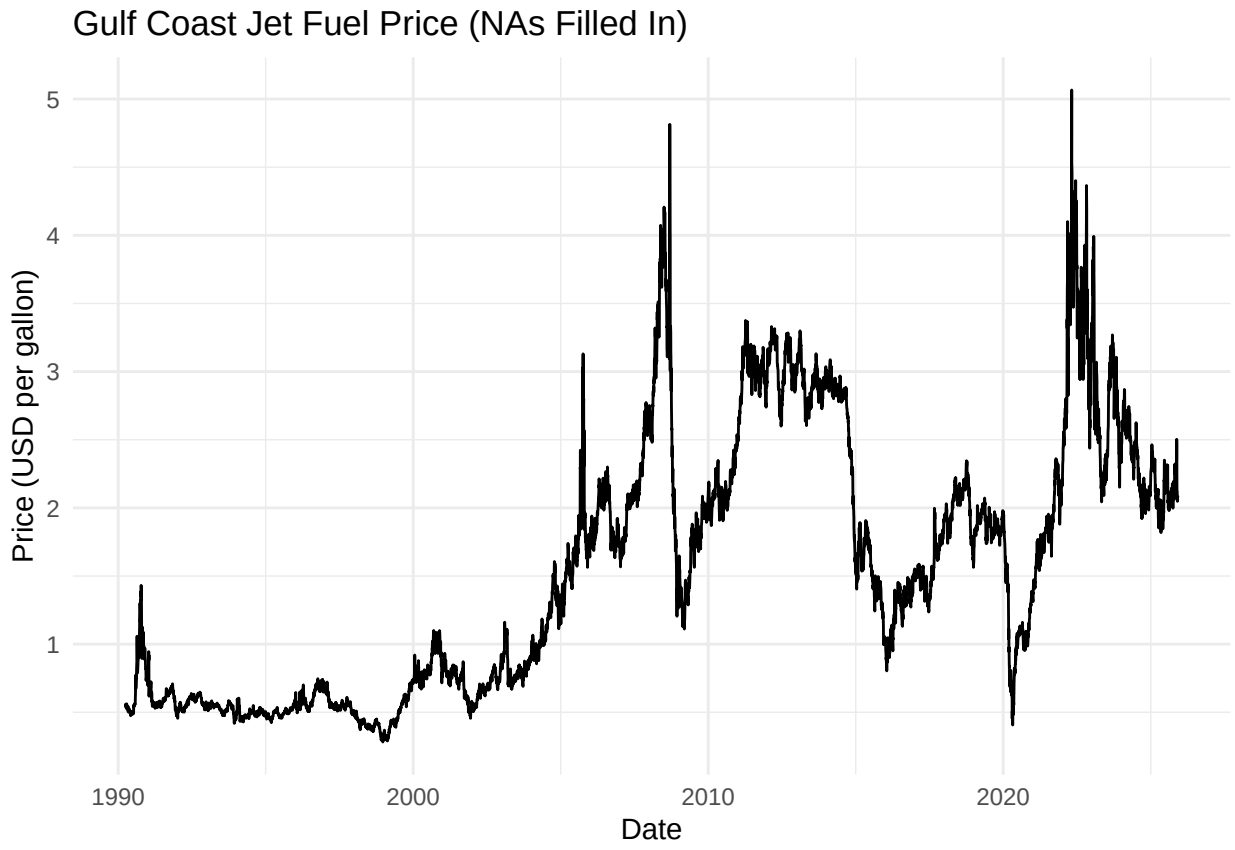
```r
fuel_raw$observation_date <- as.Date(fuel_raw$observation_date)
names(fuel_raw)[names(fuel_raw) == "DJFUELUSGULF"] <- "Fuel"

fuel_ts <- as_tsibble(fuel_raw, index = observation_date) |>
  fill_gaps() |>
  mutate(Fuel = forecast::na.interp(Fuel))

N <- nrow(fuel_ts)
train_size <- floor(0.9 * N)

fuel_train <- fuel_ts[1:train_size, ]
fuel_test  <- fuel_ts[(train_size + 1):N, ]
h <- nrow(fuel_test)

ggplot(as.data.frame(fuel_raw),
       aes(x = observation_date, y = Fuel)) +
  geom_line(color = "black") +
  labs(title = "Gulf Coast Jet Fuel Price (Unaltered)",
       x = "Date", y = "Price (USD per gallon)") +
  theme_minimal()
```




```r
ggplot(as.data.frame(fuel_ts),
       aes(x = observation_date, y = Fuel)) +
  geom_line(color = "black") +
  labs(title = "Gulf Coast Jet Fuel Price (NAs Filled In)",
```

```
       x = "Date", y = "Price (USD per gallon)") +
  theme_minimal()
```

```
## Don't know how to automatically pick scale for object of type <ts>. Defaulting
## to continuous.
```



The above graphs show the time series of oil barrel prices in the Gulf since 1990. Graph one is unaltered, and graph two has the NAs taken care of. This is common practice in the world of forecasting.

Forecast 1: ARIMA Model

```
fit_arima <- fuel_train |>
  model(
    arima = ARIMA(
      Fuel ~ trend() + fourier("year", K = 5)
    )
  )

report(fit_arima)
```

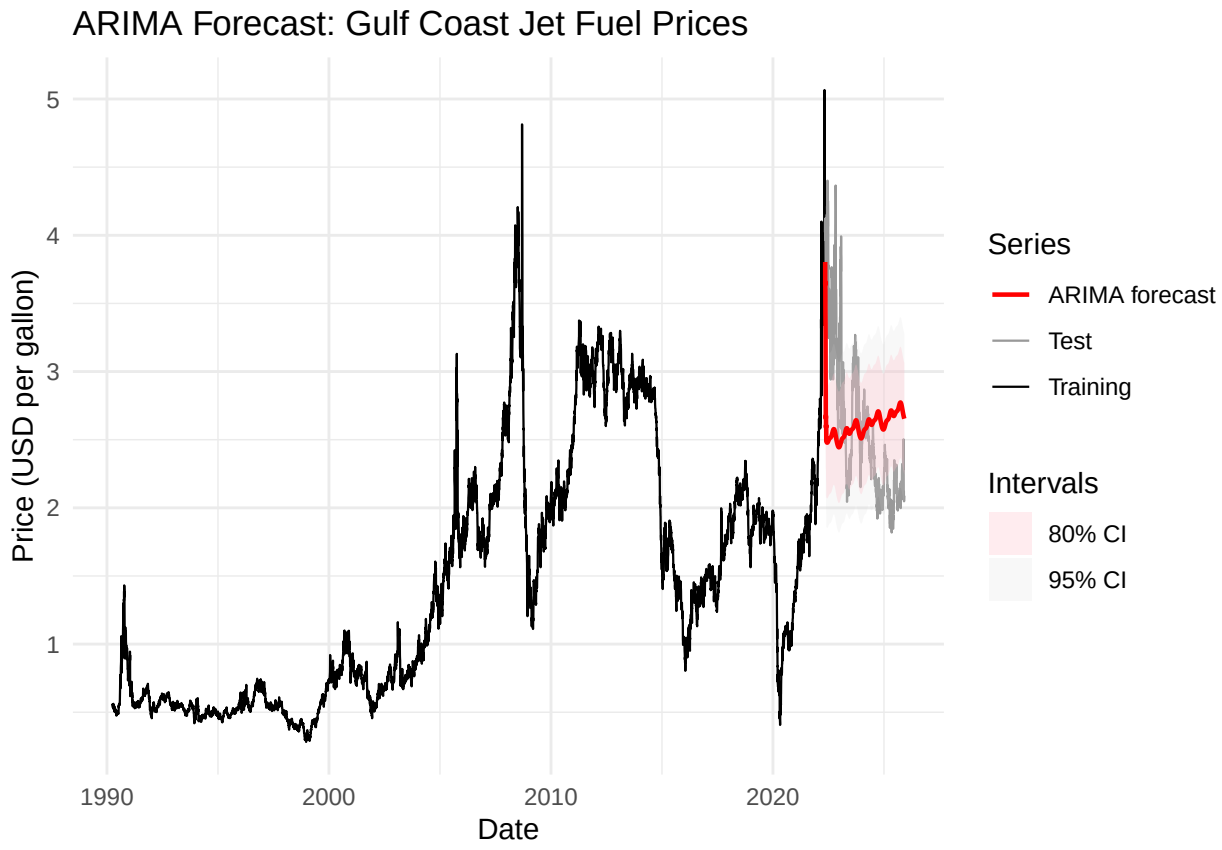
```
## Series: Fuel
## Model: LM w/ ARIMA(0,0,4)(0,0,2)[7] errors
##
```

```
## Coefficients:
##          ma1     ma2     ma3     ma4    sma1    sma2  trend()
##       1.8531  2.1219  1.6542  0.7736  0.8601  0.4616    2e-04
## s.e.  0.0070  0.0119  0.0101  0.0053  0.0090  0.0071    0e+00
##       fourier("year", K = 5)C1_365  fourier("year", K = 5)S1_365
##                            -0.0365                       -0.0142
## s.e.                        0.0146                        0.0145
##       fourier("year", K = 5)C2_365  fourier("year", K = 5)S2_365
##                            -0.0379                        0.0036
## s.e.                        0.0144                        0.0144
##       fourier("year", K = 5)C3_365  fourier("year", K = 5)S3_365
##                             0.0025                        0.0104
## s.e.                        0.0140                        0.0140
##       fourier("year", K = 5)C4_365  fourier("year", K = 5)S4_365
##                             0.0023                        0.0098
## s.e.                        0.0136                        0.0136
##       fourier("year", K = 5)C5_365  fourier("year", K = 5)S5_365  intercept
##                            -0.0049                       -0.0048     0.3924
## s.e.                        0.0131                        0.0131     0.0207
##
## sigma^2 estimated as 0.004249:  log likelihood=15382.53
## AIC=-30727.06   AICc=-30726.99   BIC=-30587.04
```

```
fc_arima <- fit_arima |>
  forecast(h = h)

fc_arima_df <- fc_arima |>
  mutate(
    .mean   = sapply(Fuel, mean),
    lower80 = sapply(Fuel, quantile, p = 0.10),
    upper80 = sapply(Fuel, quantile, p = 0.90),
    lower95 = sapply(Fuel, quantile, p = 0.025),
    upper95 = sapply(Fuel, quantile, p = 0.975)
  ) |>
  as.data.frame()

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_ribbon(data = fc_arima_df,
              aes(x = observation_date, ymin = lower95, ymax = upper95,
                  fill = "95% CI"),
              alpha = 0.15, inherit.aes = FALSE) +
  geom_ribbon(data = fc_arima_df,
              aes(x = observation_date, ymin = lower80, ymax = upper80,
                  fill = "80% CI"),
              alpha = 0.30, inherit.aes = FALSE) +
  geom_line(data = fc_arima_df,
            aes(x = observation_date, y = .mean, colour = "ARIMA forecast"),
            size = 0.8, na.rm = TRUE) +
```

```
  labs(title = "ARIMA Forecast: Gulf Coast Jet Fuel Prices",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "ARIMA forecast" = "red")) +
  scale_fill_manual(values = c("80% CI" = "pink",
                               "95% CI" = "lightgrey")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"),
         fill   = guide_legend(title = "Intervals"))
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



The Arima forecast model, which is ARIMA(0,0,4)(0,0,2)[7] containing seasonality and trend, does a good job predicting the oil price drop, but then predicts a steady increase of oil prices that echo earlier trends. It did not think that it would take so long for prices to recover. It covers the rough shape of the graph well, but does not do a great job predicting values and the trend is incorrect.

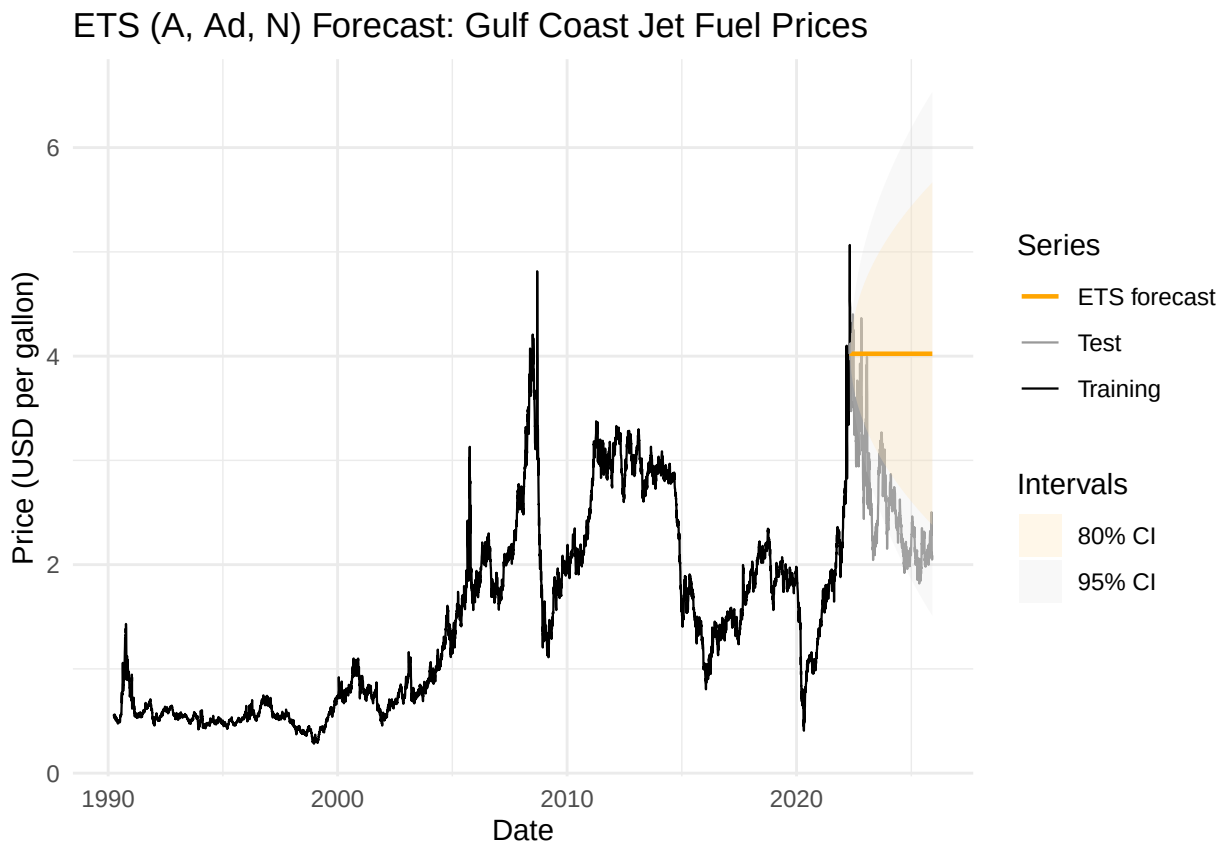
Forecast 2: ETS Model

```
fit_ets <- fuel_train |>
  model(
    ets = ETS(Fuel ~ error("A") + trend("Ad") + season("N"))
  )

report(fit_ets)
```

```
## Series: Fuel
## Model: ETS(A,Ad,N)
##   Smoothing parameters:
##     alpha = 0.9998997
##     beta  = 0.003210508
##     phi   = 0.8000001
##
##   Initial states:
##       l[0]       b[0]
##  0.6162336 -0.1853616
##
##   sigma^2:  0.0012
##
##      AIC     AICc      BIC
## 31299.43 31299.44 31343.65
```

```
fc_ets <- fit_ets |>
  forecast(h = h)

fc_ets_df <- fc_ets |>
  mutate(
    .mean   = sapply(Fuel, mean),
    lower80 = sapply(Fuel, function(z) quantile(z, p = 0.10)),
    upper80 = sapply(Fuel, function(z) quantile(z, p = 0.90)),
    lower95 = sapply(Fuel, function(z) quantile(z, p = 0.025)),
    upper95 = sapply(Fuel, function(z) quantile(z, p = 0.975))
  ) |>
  as.data.frame()

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_ribbon(data = fc_ets_df,
              aes(x = observation_date, ymin = lower95, ymax = upper95,
                  fill = "95% CI"),
              alpha = 0.15, inherit.aes = FALSE) +
  geom_ribbon(data = fc_ets_df,
              aes(x = observation_date, ymin = lower80, ymax = upper80,
                  fill = "80% CI"),
```

```
              alpha = 0.30, inherit.aes = FALSE) +
  geom_line(data = fc_ets_df,
            aes(x = observation_date, y = .mean, colour = "ETS forecast"),
            size = 0.8, na.rm = TRUE) +
  labs(title = "ETS (A, Ad, N) Forecast: Gulf Coast Jet Fuel Prices",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "ETS forecast" = "orange")) +
  scale_fill_manual(values = c("80% CI" = "moccasin",
                               "95% CI" = "lightgrey")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"),
         fill   = guide_legend(title = "Intervals"))
```



The ETS forecast, which used dampening but no seasonality, does a poor job predicting the testing data. It has no trend or seasonality, and in some places the testing data doesn't even fall in the 80% CI.

Forecast 3: Holt-Winters

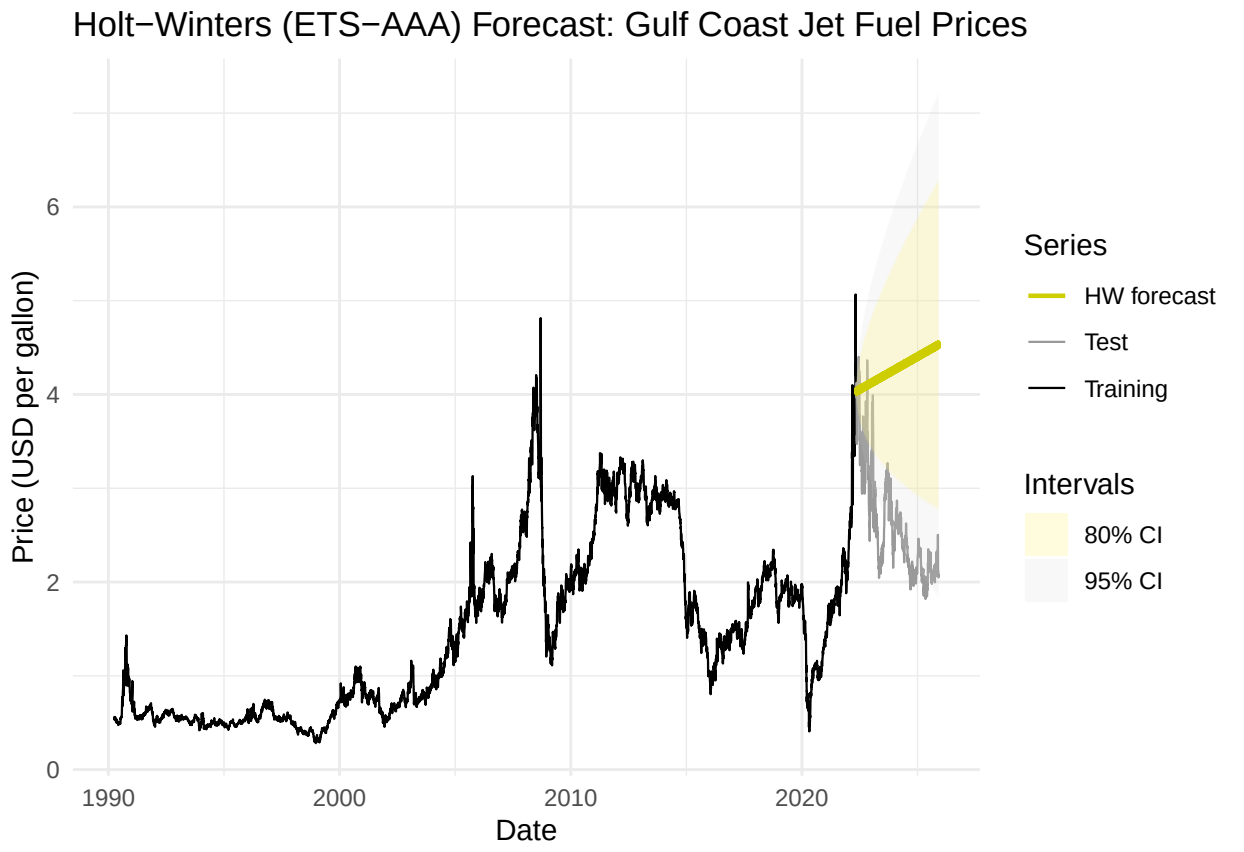
```
fit_hw <- fuel_train |>
  model(
    hw = ETS(Fuel ~ error("A") + trend("A") + season("A"))
  )
```

```r
report(fit_hw)
```

```
## Series: Fuel
## Model: ETS(A,A,A)
##   Smoothing parameters:
##     alpha = 0.9508258
##     beta  = 0.000140905
##     gamma = 0.04473221
##
##   Initial states:
##       l[0]         b[0]      s[0]      s[-1]       s[-2]       s[-3]      s[-4]
##  0.7052686 7.475581e-05 0.0626466 0.02937759 -0.08201719 -0.01323452 0.01742088
##       s[-5]       s[-6]
##  -0.01956167 0.005368313
##
##   sigma^2:  0.0013
##
##      AIC     AICc      BIC
## 31995.94 31995.97 32084.37
```

```r
fc_hw <- fit_hw |>
  forecast(h = h)

fc_hw_df <- fc_hw |>
  mutate(
    .mean   = sapply(Fuel, mean),
    lower80 = sapply(Fuel, quantile, p = 0.10),
    upper80 = sapply(Fuel, quantile, p = 0.90),
    lower95 = sapply(Fuel, quantile, p = 0.025),
    upper95 = sapply(Fuel, quantile, p = 0.975)
  ) |>
  as.data.frame()

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_ribbon(data = fc_hw_df,
              aes(x = observation_date, ymin = lower95, ymax = upper95,
                  fill = "95% CI"),
              alpha = 0.15, inherit.aes = FALSE) +
  geom_ribbon(data = fc_hw_df,
              aes(x = observation_date, ymin = lower80, ymax = upper80,
                  fill = "80% CI"),
              alpha = 0.30, inherit.aes = FALSE) +
  geom_line(data = fc_hw_df,
            aes(x = observation_date, y = .mean, colour = "HW forecast"),
            size = 0.8, na.rm = TRUE) +
  labs(title = "Holt-Winters (ETS-AAA) Forecast: Gulf Coast Jet Fuel Prices",
```

```
      x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "HW forecast" = "yellow3")) +
  scale_fill_manual(values = c("80% CI" = "khaki1",
                               "95% CI" = "lightgrey")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"),
         fill   = guide_legend(title = "Intervals"))
```



Holt Winters actually does a worse job than the ETS prediction, which was just a horizonal line. It relies too heavily on the historical trend and completely misses the downturn that Arima caught. Because of the added seasonal variance in this forecast, the model has wider confidence bands.

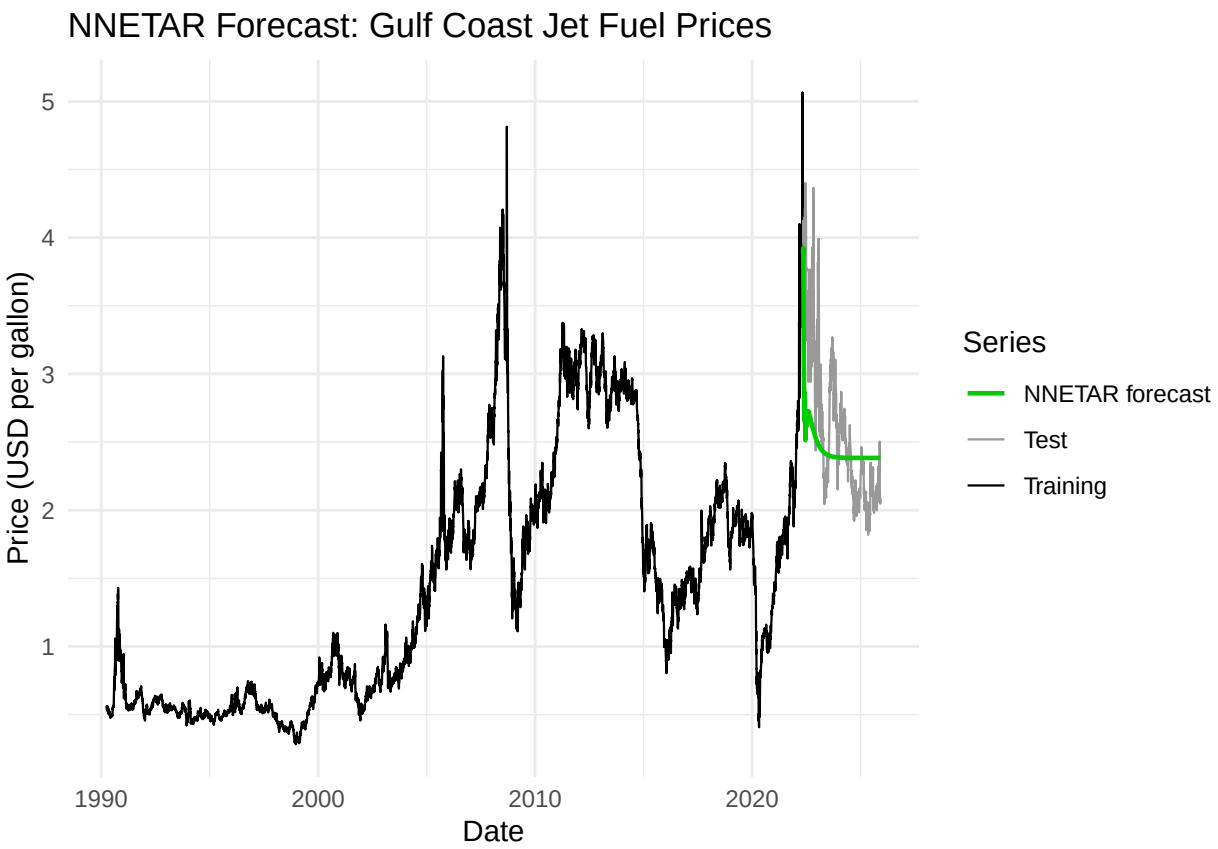
Forecast 4: NNETAR

```
library(forecast)

set.seed(817)

h_nnet <- nrow(fuel_test)

train_ts <- ts(fuel_train$Fuel, frequency = 7)
```

```r
fit_nnetar <- forecast::nnetar(train_ts, repeats = 5)

summary(fit_nnetar)
```

```
##           Length Class        Mode
## x         11725  ts           numeric
## m             1  -none-       numeric
## p             1  -none-       numeric
## P             1  -none-       numeric
## scalex        2  -none-       list
## size          1  -none-       numeric
## subset    11725  -none-       numeric
## model         5  nnetarmodels list
## nnetargs      0  -none-       list
## fitted    11725  ts           numeric
## residuals 11725  ts           numeric
## lags         40  -none-       numeric
## series        1  -none-       character
## method        1  -none-       character
## call          3  -none-       call
```

```r
fc_nnetar_raw <- forecast::forecast(
  fit_nnetar,
  h  = h_nnet,
  PI = FALSE
)

fc_nnetar_df <- data.frame(
  observation_date = tail(fuel_ts$observation_date, h_nnet),
  .mean            = as.numeric(fc_nnetar_raw$mean)
)

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = fc_nnetar_df,
            aes(x = observation_date, y = .mean, colour = "NNETAR forecast"),
            size = 0.8, na.rm = TRUE) +
  labs(title = "NNETAR Forecast: Gulf Coast Jet Fuel Prices",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "NNETAR forecast" = "green3")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"))
```



This NNETAR model (Neural Network Auto Regression) is simplified due to compute limitations, but manages to predict the downturn just like Arima and unlike ETS and HW. It slightly factors in the seasonal spikes. Unlike the Arima model, it matches the shape of the test data for longer a longer period of time. The NNETAR model has the steep decline in price leveling out instead of continuing to fall, but there is no price rise, which is accurate to the test data. A stronger computer would have been able to generate a more complex and accurate neural network.

It is important to note that there are no CI bands here due to an R package issue and the forecast:: method taking too long.

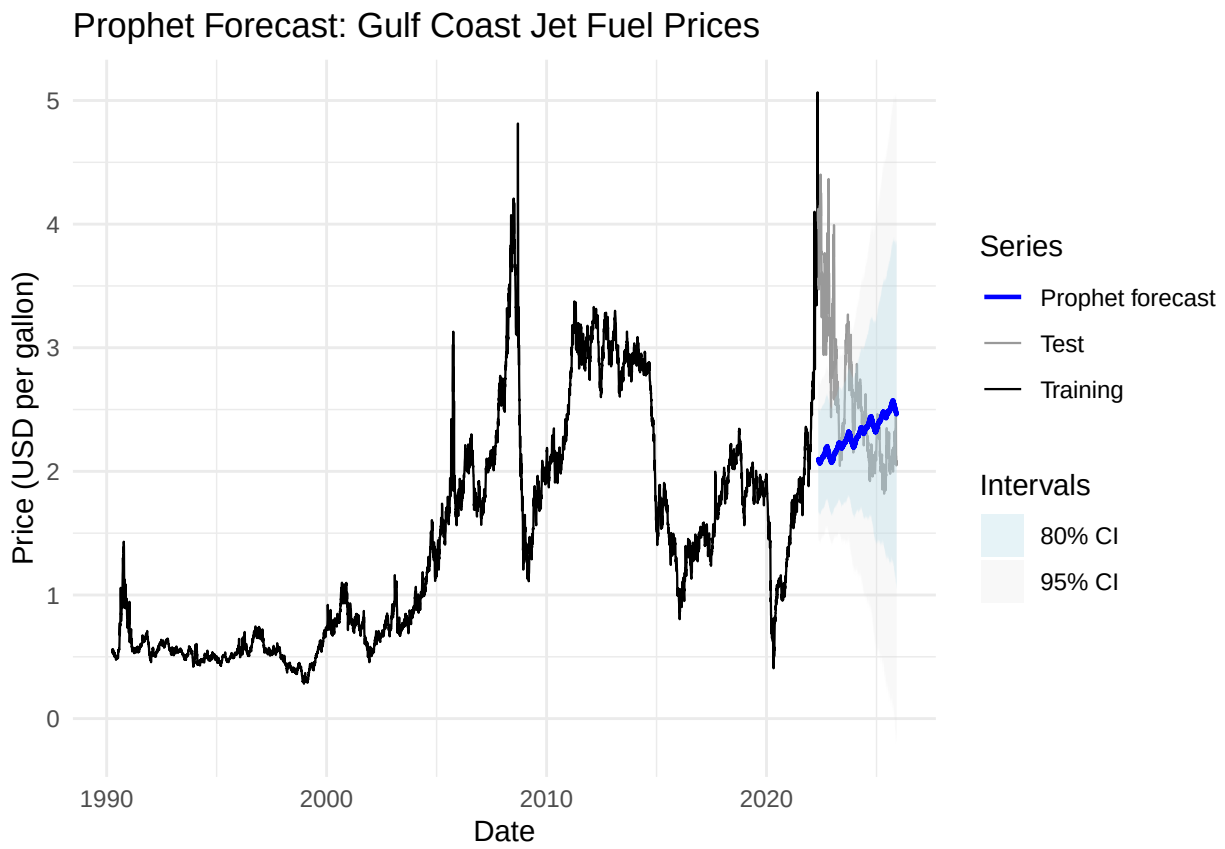
Forecast 5: Prophet

```
fit_prophet <- fuel_train |>
  model(
    prophet = prophet(
      Fuel ~
        season(period = "week", order = 5) +
        season(period = "year", order = 5)
    )
  )

report(fit_prophet)
```

```
## Series: Fuel
```

```
## Model: prophet
##
## A model specific report is not available for this model class.
```

```
fc_prophet <- fit_prophet |>
  forecast(h = h)


fc_prophet_df <- fc_prophet |>
  mutate(
    .mean   = sapply(Fuel, mean),
    lower80 = sapply(Fuel, function(z) quantile(z, p = 0.10)),
    upper80 = sapply(Fuel, function(z) quantile(z, p = 0.90)),
    lower95 = sapply(Fuel, function(z) quantile(z, p = 0.025)),
    upper95 = sapply(Fuel, function(z) quantile(z, p = 0.975))
  ) |>
  as.data.frame()

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_ribbon(data = fc_prophet_df,
              aes(x = observation_date, ymin = lower95, ymax = upper95,
                  fill = "95% CI"),
              alpha = 0.15, inherit.aes = FALSE) +
  geom_ribbon(data = fc_prophet_df,
              aes(x = observation_date, ymin = lower80, ymax = upper80,
                  fill = "80% CI"),
              alpha = 0.30, inherit.aes = FALSE) +
  geom_line(data = fc_prophet_df,
            aes(x = observation_date, y = .mean, colour = "Prophet forecast"),
            size = 0.8, na.rm = TRUE) +
  labs(title = "Prophet Forecast: Gulf Coast Jet Fuel Prices",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "Prophet forecast" = "blue")) +
  scale_fill_manual(values = c("80% CI" = "lightblue",
                               "95% CI" = "lightgrey")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"),
         fill   = guide_legend(title = "Intervals"))
```

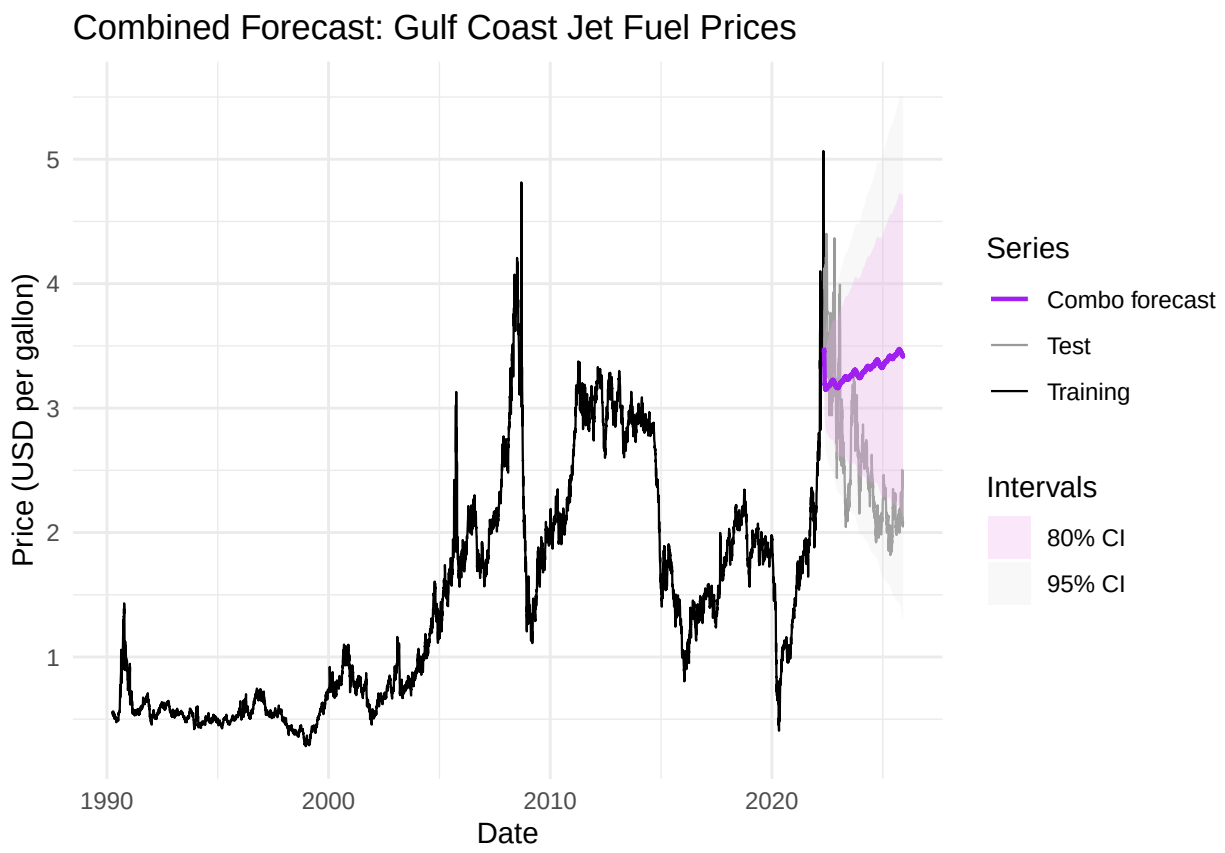


At its core, the Prophet model is very similar to the Arima model. Both use trend, seasonality, and cycles to predict future data. In our case here, though, the Arima model did a better job. They both have an upward trend, but Prophet doesn't account for the slow drop off Arima caught. Interestingly, almost all of the data post-2022 falls well within the CI bands.

Forecast 6: Combination

```
fc_combo_df <- data.frame(
  observation_date = fc_arima_df$observation_date,
  .mean   = rowMeans(cbind(fc_arima_df$.mean,
                           fc_ets_df$.mean,
                           fc_hw_df$.mean,
                           fc_prophet_df$.mean), na.rm = TRUE),
  lower80 = rowMeans(cbind(fc_arima_df$lower80,
                           fc_ets_df$lower80,
                           fc_hw_df$lower80,
                           fc_prophet_df$lower80), na.rm = TRUE),
  upper80 = rowMeans(cbind(fc_arima_df$upper80,
                           fc_ets_df$upper80,
                           fc_hw_df$upper80,
                           fc_prophet_df$upper80), na.rm = TRUE),
  lower95 = rowMeans(cbind(fc_arima_df$lower95,
                           fc_ets_df$lower95,
                           fc_hw_df$lower95,
                           fc_prophet_df$lower95), na.rm = TRUE),
```

```
  upper95 = rowMeans(cbind(fc_arima_df$upper95,
                           fc_ets_df$upper95,
                           fc_hw_df$upper95,
                           fc_prophet_df$upper95), na.rm = TRUE)
)

ggplot() +
  geom_line(data = as.data.frame(fuel_train),
            aes(x = observation_date, y = Fuel, colour = "Training"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = as.data.frame(fuel_test),
            aes(x = observation_date, y = Fuel, colour = "Test"),
            size = 0.4, na.rm = TRUE) +
  geom_ribbon(data = fc_combo_df,
              aes(x = observation_date, ymin = lower95, ymax = upper95,
                  fill = "95% CI"),
              alpha = 0.15, inherit.aes = FALSE) +
  geom_ribbon(data = fc_combo_df,
              aes(x = observation_date, ymin = lower80, ymax = upper80,
                  fill = "80% CI"),
              alpha = 0.30, inherit.aes = FALSE) +
  geom_line(data = fc_combo_df,
            aes(x = observation_date, y = .mean, colour = "Combo forecast"),
            size = 0.8, na.rm = TRUE) +
  labs(title = "Combined Forecast: Gulf Coast Jet Fuel Prices",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c("Training" = "black",
                                 "Test" = "grey60",
                                 "Combo forecast" = "purple")) +
  scale_fill_manual(values = c("80% CI" = "plum2",
                               "95% CI" = "lightgrey")) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series"),
         fill   = guide_legend(title = "Intervals"))
```



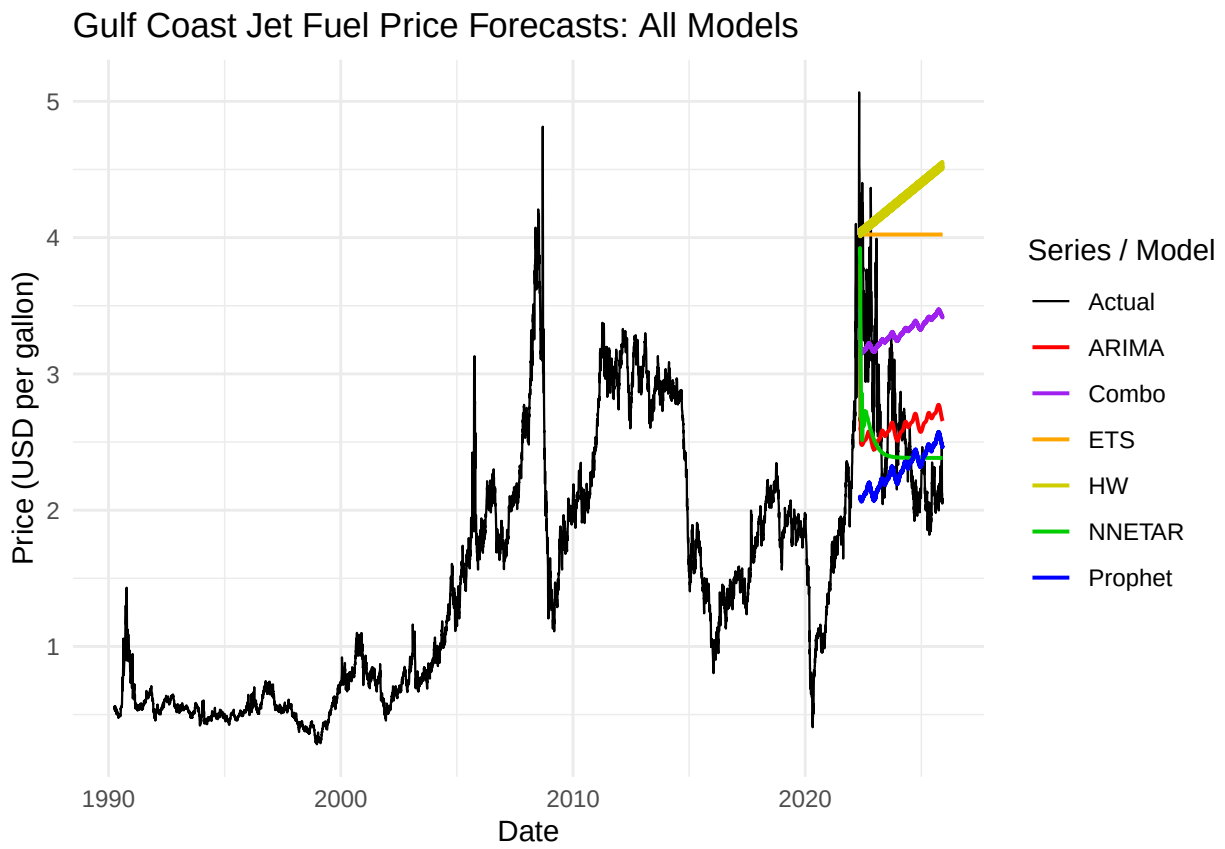
The above combination model takes a straight average of the prior 5 models. It is very similar to the prophet model, and has the same upwards trend due to its constant appearance in other models. It catches the slight drop off noticed in Arima and NNETAR, but it isn't as pronounced due to the inclusion of other, non-drop off models.

All Models

```
all_fc_df <- rbind(
  data.frame(
    observation_date = fc_arima_df$observation_date,
    value            = fc_arima_df$.mean,
    model            = "ARIMA"
  ),
  data.frame(
    observation_date = fc_ets_df$observation_date,
    value            = fc_ets_df$.mean,
    model            = "ETS"
  ),
  data.frame(
    observation_date = fc_hw_df$observation_date,
    value            = fc_hw_df$.mean,
    model            = "HW"
  ),
  data.frame(
```

```
    observation_date = fc_nnetar_df$observation_date,
    value            = fc_nnetar_df$.mean,
    model            = "NNETAR"
  ),
  data.frame(
    observation_date = fc_prophet_df$observation_date,
    value            = fc_prophet_df$.mean,
    model            = "Prophet"
  ),
  data.frame(
    observation_date = fc_combo_df$observation_date,
    value            = fc_combo_df$.mean,
    model            = "Combo"
  )
)

ggplot() +
  geom_line(data = as.data.frame(fuel_ts),
            aes(x = observation_date, y = Fuel, colour = "Actual"),
            size = 0.4, na.rm = TRUE) +
  geom_line(data = all_fc_df,
            aes(x = observation_date, y = value, colour = model),
            size = 0.7, na.rm = TRUE) +
  labs(title = "Gulf Coast Jet Fuel Price Forecasts: All Models",
       x = "Date", y = "Price (USD per gallon)") +
  scale_colour_manual(values = c(
    "Actual"  = "black",
    "ARIMA"   = "red",
    "ETS"     = "orange",
    "HW"      = "yellow3",
    "NNETAR"  = "green3",
    "Prophet" = "blue",
    "Combo"   = "purple"
  )) +
  theme_minimal() +
  guides(colour = guide_legend(title = "Series / Model"))
```

```
## Don't know how to automatically pick scale for object of type <ts>. Defaulting
## to continuous.
```



This graph layers all of the previous models on top of each other. There isn't much new to talk about, other than the fact that visually speaking, NNETAR definitely follows the actual data closest. Arima looks good too, as it's the only other model that predicted the price drop, but it overpredicted a bounce back. NNETAR did not anticipate that prices would rise, making it look closest to the test data.

Errors

```
arima_df <- merge(as.data.frame(fuel_test),
                  fc_arima_df[, c("observation_date", ".mean")],
                  by = "observation_date", all.x = TRUE)
e_arima <- arima_df$Fuel - arima_df$.mean
rmse_arima <- sqrt(mean(e_arima^2, na.rm = TRUE))
mae_arima  <- mean(abs(e_arima), na.rm = TRUE)
mape_arima <- mean(abs(e_arima / arima_df$Fuel), na.rm = TRUE) * 100
mse_arima  <- mean(e_arima^2, na.rm = TRUE)

ets_df <- merge(as.data.frame(fuel_test),
                fc_ets_df[, c("observation_date", ".mean")],
                by = "observation_date", all.x = TRUE)
e_ets <- ets_df$Fuel - ets_df$.mean
rmse_ets <- sqrt(mean(e_ets^2, na.rm = TRUE))
mae_ets  <- mean(abs(e_ets), na.rm = TRUE)
mape_ets <- mean(abs(e_ets / ets_df$Fuel), na.rm = TRUE) * 100
mse_ets  <- mean(e_ets^2, na.rm = TRUE)
```

```r
hw_df <- merge(as.data.frame(fuel_test),
               fc_hw_df[, c("observation_date", ".mean")],
               by = "observation_date", all.x = TRUE)
e_hw <- hw_df$Fuel - hw_df$.mean
rmse_hw <- sqrt(mean(e_hw^2, na.rm = TRUE))
mae_hw  <- mean(abs(e_hw), na.rm = TRUE)
mape_hw <- mean(abs(e_hw / hw_df$Fuel), na.rm = TRUE) * 100
mse_hw  <- mean(e_hw^2, na.rm = TRUE)

nnetar_df <- merge(as.data.frame(fuel_test),
                   fc_nnetar_df[, c("observation_date", ".mean")],
                   by = "observation_date", all.x = TRUE)
e_nnetar <- nnetar_df$Fuel - nnetar_df$.mean
rmse_nnetar <- sqrt(mean(e_nnetar^2, na.rm = TRUE))
mae_nnetar  <- mean(abs(e_nnetar), na.rm = TRUE)
mape_nnetar <- mean(abs(e_nnetar / nnetar_df$Fuel), na.rm = TRUE) * 100
mse_nnetar  <- mean(e_nnetar^2, na.rm = TRUE)

prophet_df <- merge(as.data.frame(fuel_test),
                    fc_prophet_df[, c("observation_date", ".mean")],
                    by = "observation_date", all.x = TRUE)
e_prophet <- prophet_df$Fuel - prophet_df$.mean
rmse_prophet <- sqrt(mean(e_prophet^2, na.rm = TRUE))
mae_prophet  <- mean(abs(e_prophet), na.rm = TRUE)
mape_prophet <- mean(abs(e_prophet / prophet_df$Fuel), na.rm = TRUE) * 100
mse_prophet  <- mean(e_prophet^2, na.rm = TRUE)

combo_df <- merge(as.data.frame(fuel_test),
                  fc_combo_df[, c("observation_date", ".mean")],
                  by = "observation_date", all.x = TRUE)
e_combo <- combo_df$Fuel - combo_df$.mean
rmse_combo <- sqrt(mean(e_combo^2, na.rm = TRUE))
mae_combo  <- mean(abs(e_combo), na.rm = TRUE)
mape_combo <- mean(abs(e_combo / combo_df$Fuel), na.rm = TRUE) * 100
mse_combo  <- mean(e_combo^2, na.rm = TRUE)


library(fabletools)

g_arima   <- fabletools::glance(fit_arima)
g_ets     <- fabletools::glance(fit_ets)
g_hw      <- fabletools::glance(fit_hw)
g_prophet <- fabletools::glance(fit_prophet)

AIC_arima <- if ("AIC" %in% names(g_arima)) g_arima$AIC else NA_real_
BIC_arima <- if ("BIC" %in% names(g_arima)) g_arima$BIC else NA_real_

AIC_ets   <- if ("AIC" %in% names(g_ets))   g_ets$AIC   else NA_real_
BIC_ets   <- if ("BIC" %in% names(g_ets))   g_ets$BIC   else NA_real_

AIC_hw    <- if ("AIC" %in% names(g_hw))    g_hw$AIC    else NA_real_
BIC_hw    <- if ("BIC" %in% names(g_hw))    g_hw$BIC    else NA_real_
```

```
AIC_prophet <- if ("AIC" %in% names(g_prophet)) g_prophet$AIC else NA_real_
BIC_prophet <- if ("BIC" %in% names(g_prophet)) g_prophet$BIC else NA_real_

AIC_nnetar <- tryCatch(AIC(fit_nnetar), error = function(e) NA_real_)
BIC_nnetar <- tryCatch(BIC(fit_nnetar), error = function(e) NA_real_)

AIC_combo <- NA_real_
BIC_combo <- NA_real_


model_errors <- rbind(
  data.frame(Model = "ARIMA",
             RMSE = rmse_arima,   MAE = mae_arima,
             MAPE = mape_arima,   MSE = mse_arima,
             AIC = AIC_arima,     BIC = BIC_arima),
  data.frame(Model = "ETS",
             RMSE = rmse_ets,     MAE = mae_ets,
             MAPE = mape_ets,     MSE = mse_ets,
             AIC = AIC_ets,       BIC = BIC_ets),
  data.frame(Model = "Holt-Winters",
             RMSE = rmse_hw,      MAE = mae_hw,
             MAPE = mape_hw,      MSE = mse_hw,
             AIC = AIC_hw,        BIC = BIC_hw),
  data.frame(Model = "NNETAR",
             RMSE = rmse_nnetar,  MAE = mae_nnetar,
             MAPE = mape_nnetar,  MSE = mse_nnetar,
             AIC = AIC_nnetar,    BIC = BIC_nnetar),
  data.frame(Model = "Prophet",
             RMSE = rmse_prophet, MAE = mae_prophet,
             MAPE = mape_prophet, MSE = mse_prophet,
             AIC = AIC_prophet,   BIC = BIC_prophet),
  data.frame(Model = "Combination",
             RMSE = rmse_combo,   MAE = mae_combo,
             MAPE = mape_combo,   MSE = mse_combo,
             AIC = AIC_combo,     BIC = BIC_combo)
)

model_errors <- as.data.frame(model_errors)
rownames(model_errors) <- NULL
model_errors
```

```
##          Model      RMSE       MAE     MAPE       MSE       AIC       BIC
## 1        ARIMA 0.6073493 0.4947730 19.09351 0.3688731 -30727.06 -30587.04
## 2          ETS 1.5394602 1.4410316 62.01040 2.3699377  31299.43  31343.65
## 3 Holt-Winters 1.8259403 1.6994076 73.33318 3.3340579  31995.94  32084.37
## 4       NNETAR 0.4949070 0.3731192 13.45515 0.2449329        NA        NA
## 5      Prophet 0.7301940 0.5410053 18.90077 0.5331832        NA        NA
## 6  Combination 0.9507600 0.8510960 36.96617 0.9039445        NA        NA
```

The NNETAR model leads in all 4 of its comparative categories. It has the lowest RMSE, MAE, MAPE, and MSE, solidifying it's place as the most accurate model. Arima has the lowest AIC and BIC, solidifying it as more accurate than ETS and HW. It is also consistenly second in the other comparative stats, narrowly edging out the Prophet model.

There is no AIC/BIC figure for NNETAR, Prophet, and the combination model because none of them are likelihood models, but that does not stop NNETAR from clearly being the most accurate model of the collection, as well as visually.

Conclusions and Future Work

Can talk about how there were some missing values in original time series that we auto filled in also can bring up how we did a 90/10 test/train split, and that 90 happened to be at the peak making predictions tougher, further exploration could be trying out with a smaller training subset NNETAR is simplified due to compute limitations

References

The dataset we used for this project (DJFUELUSGULF) was sourced from FRED and can be found here: https://fred.stlouisfed.org/series/DJFUELUSGULF. No other sources were used.